# DriftGuard: A Diagnostic Study of Train-Free Language-Drift Control

**Anonymous submission**


## Abstract

Unintended language switching is a practical failure mode for multilingual language models: a response can begin in the requested language and then drift into another language, or suppress legitimate code-switching while trying to enforce a target language. We study DriftGuard, a train-free language-control workflow that monitors generation state and applies target-language repair only under a drift-risk gate. A completed three-family evaluation on Language Confusion Benchmark, CodeMixQA, and FLORES-200 with Qwen/Qwen2.5-7B-Instruct through a vLLM backend produces a mixed diagnostic result rather than a positive mitigation headline. Full DriftGuard trails a plain target-language instruction on CodeMixQA (0.0125 versus 0.0250 mean score) and FLORES-200 (0.434874 versus 0.446445), while its LCB score is language-adherence evidence under a severe truncation confound rather than semantic-preservation evidence. The best observed condition is family-specific: monitor-only on LCB, language-lock prompting on FLORES-200, and monitor-only near floor on CodeMixQA. These findings turn the method into an audit instrument: the framework exposes where language-control mechanisms fail, which comparisons are necessary, and why future drift mitigation must separate target-language adherence, semantic preservation, truncation, and legitimate code-switching before claiming broad robustness.


## Introduction

Multilingual language models are often asked to follow a user-specified output language under mixed prompts, translated context, named entities, and code-switched evidence. The failure mode studied here is not ordinary mistranslation. A model may start in the target language, copy source-language spans, continue in English, or produce a hybrid response that is neither a faithful code-switched answer nor a target-language answer. Prior work frames this behavior as language confusion, unintended code-switching, or weak language-form control, and shows that it is measurable across prompts and language families (Marchisio et al. 2024; Chen et al. 2024b; Zhang et al. 2023). Other work argues that the behavior is connected to internal language-form representations and to train-free control mechanisms (Nie, Schmid, and Schutze 2025; Lopo et al. 2025; Goncharov, Kondusov, and Zaytsev 2025; Zhong et al. 2025). These observations motivate a controller that can intervene during generation rather than only rewriting the initial prompt.

The simple baseline is also strong. If a plain instruction to answer only in the target language already reduces drift, a more elaborate controller must justify its gate, its intervention, and its failure modes. The plan for DriftGuard therefore included a plain target-language instruction, a stronger language-lock prompt, a published-style Adaptive LATB baseline, and three DriftGuard ablations. This comparison is essential because decoding-time and prompt-time methods are active competitors for language confusion mitigation (Lee et al. 2025; Ukarapol, Sarapat, and Chukamphaeng 2026; Choo et al. 2026). It also prevents the paper from treating any improvement over an unmodified mixed-context prompt as evidence that the proposed dynamic gate is useful.

This paper reports a full diagnostic matrix rather than a success claim. The evaluation contains three benchmark families, eight conditions, 80 tasks per family-condition cell, and 1,920 scored rows. It uses public benchmark families that stress different aspects of the problem: LCB for target-language adherence under language-confusion prompts, CodeMixQA for legitimate code-switching and answer preservation, and FLORES-200 for semantic preservation with professional translation references. The evaluated backend is Qwen/Qwen2.5-7B-Instruct through vLLM with a fixed 128-new-token generation budget. Results that contradict the planned mitigation hypothesis are reported as failure analysis rather than rewritten as positive contributions.

This framing matters because language-control papers are easy to overstate. A method can improve over an unmodified mixed-context prompt while still losing to a one-sentence target-language instruction; it can also reduce wrong-language spans while increasing truncation or losing answer semantics. We therefore treat negative comparisons as first-class evidence: a claim appears in contribution language only when it survives the stronger plain-prompt and ablation comparisons.

The main result is negative in the sense that the current full DriftGuard recipe is not a robust winner. On LCB, full DriftGuard reaches 0.8375 mean score, but the family is dominated by truncation and does not establish semantic correctness under this budget; plain target instruction reaches 0.8500 and monitor-only reaches 0.8625. On FLORES-200, full DriftGuard reaches 0.434874, be-

low plain target instruction at 0.446445 and language-lock prompting at 0.455586. On CodeMixQA, full DriftGuard reaches 0.0125 while plain target instruction reaches 0.0250 and the family remains near floor. These results contradict a broad claim that full DriftGuard improves over plain target instruction across the executed families, and they reject a strong claim that the current run demonstrates preservation of code-switched answer correctness.

The contribution is therefore a calibrated one. First, we give a workflow for train-free drift monitoring and intervention, with ablations that isolate whether a dynamic gate adds value. Second, we report a complete three-family evidence matrix that includes a simple prompt baseline, a nontrivial train-free baseline, and DriftGuard variants. Third, we show that the current dynamic gate can underperform simpler controls and that CodeMixQA exposes a severe evaluation or generation failure. Finally, we make the contradicted hypotheses explicit findings, not hidden limitations. This names the mechanism that failed: not language control in general, but the present gate and its interaction with task-specific scoring.

## Related Work

Language confusion has recently become a direct evaluation target for multilingual LLMs. The Language Confusion Benchmark formalizes the wrong-language output problem and provides prompt families for measuring whether a model follows the requested language (Marchisio et al. 2024). Complementary work studies typological and security-relevant patterns in confusion, arguing that language failures are not isolated anecdotes but structured model behavior (Chen et al. 2024b,a). Case studies in adapted multilingual models and source-language effects in in-context learning further show that confusion depends on model, language, and prompting regime rather than on one benchmark family alone (Sarapat, Ukarapol, and Hashimoto 2025; Philippy et al. 2026). Mechanistic studies further ask where confusion appears in a generation trajectory and whether internal components can be tied to language-form selection (Nie, Schmid, and Schutze 2025). DriftGuard follows this measurement-first framing but does not assume that monitoring plus intervention will improve every family.

Output-language control methods range from prompt-only constraints to token-level and representation-level interventions. Prompt-time controls are attractive because they require no model internals, but they can be hard to distinguish from a well-written target-language instruction; cross-lingual in-context learning and language steering work make that prompt/control boundary explicit (Li et al. 2023; Kirtane and Huang 2026). Recent control-oriented work treats language confusion as a mitigation problem for multilingual LLMs (Lee et al. 2025), while token-level policy optimization and language-aware token boosting explore stronger decoding or training-time interventions (Choo et al. 2026; Ukarapol, Sarapat, and Chukamphaeng 2026). Representation steering and target-language recovery methods provide closer competitors because they aim to change language form without sacrificing task behavior (Mahmoud et al. 2025; Sterz et al. 2025; Gurgurov et al. 2026). Latent-space steering, sparse-feature discovery, and sparse-dimension analyses suggest that language identity can sometimes be controlled through lower-level model signals (Goncharov, Kondusov, and Zaytsev 2025; Zhong et al. 2025; Wong, Sajjad, and Siddique 2026; Deng et al. 2025). We use these lines to motivate baselines and ablations, not to claim a new state of the art.

Code-switching work cautions that not every language mixture is an error. Multilingual LLMs are not yet reliable code-switchers (Zhang et al. 2023), but code-switching competence is a legitimate capability rather than a defect. LinCE established shared benchmark practice for linguistic code-switching tasks (Aguilar, Kar, and Solorio 2020), and recent work studies scaling code-switching data and whether LLMs can understand, reason about, and generate code-switched text (Wang et al. 2025; Winata et al. 2026; Asano, Baek, and Yamasaki 2026). Alignment, anchoring, red-teaming, and translation-oriented code-switching studies further motivate treating mixed-language behavior as a task property to evaluate rather than a surface artifact to erase (Abdaljalil et al. 2025; Park et al. 2026; Yoo, Yang, and Lee 2024; Kaji and Shah 2023). This distinction is central to the CodeMixQA result: a controller that suppresses all mixing can look good on target-language adherence while failing the intended answer behavior.

The representation literature supplies the background for why a train-free controller might be plausible. Cross-lingual pretraining and multilingual encoder work introduced shared representational spaces across languages (Devlin et al. 2018; Lample and Conneau 2019; Conneau et al. 2019), and massively multilingual sequence-to-sequence pretraining made generation-language control a practical issue (Xue et al. 2020). Later work on pretraining dynamics, isotropic transfer, cross-lingual alignment, and reference consistency sharpens the same point: language form, task knowledge, and semantic preservation can move differently across models and languages (Blevins, Gonen, and Zettlemoyer 2022; Ji et al. 2023; Ravisankar et al. 2025; Ai, Ihsani, and Kan 2025). Activation-patching, language-surgery, and latent-romanization studies suggest that semantic content and language form can sometimes be separated (Dumas et al. 2024; Lopo et al. 2025; Saji et al. 2025). DriftGuard is much more modest: it tests a surface-level monitor and repair policy around an open instruction-tuned model, and it treats failure against simple baselines as evidence about the controller rather than as a formatting inconvenience.

Benchmark design is the other relevant thread. LCB directly probes language-confusion behavior, but a method tuned only for LCB can learn to prefer target-language surface form over answer quality. CodeMixQA pushes in the opposite direction: the system must respect multilingual context and still produce the answer, so suppressing all switches is not enough. FLORES-200 adds a reference-based semantic check that makes target-script compliance insufficient. The combination is intentionally uncomfortable for a controller, because a real mitigation method should not win by exploiting one diagnostic definition. The result section shows that this multi-family design changes the story from an apparent improvement over a weak baseline to a

more precise failure analysis.

## Method

DriftGuard is a train-free language-control workflow for generation episodes in which the user or benchmark row specifies a target output language. Figure 1 gives the paper-facing pipeline. The workflow begins with an input prompt and target-language contract, then prepares the generation request under one of several conditions. During generation, a monitor summarizes language-form evidence from the partial output and the row metadata. A dynamic gate decides whether the current prefix is safe, drifting, or ambiguous. If the gate fires, the intervention path applies a bounded target-language repair instruction or lock; if it does not fire, the model continues under the original request. The output is then scored against target-language and task-specific criteria.

The evaluated conditions separate the components of the workflow. The mixed-context baseline keeps the original benchmark prompt. The plain target instruction adds only a concise requirement to answer in the target language. The language-lock prompt adds a stronger self-checking instruction. Adaptive LATB represents a train-free token-boosting style competitor in the paper-facing comparison. Full DriftGuard uses the monitor, gate, and intervention together. Three ablations isolate the mechanism: no dynamic gate applies the language-control path without waiting for the gate, monitor only observes without repair, and always-on uses a static intervention policy.

The dynamic gate is the central hypothesis under test. If it works, full DriftGuard should beat a plain target instruction and should beat or match simpler ablations because it avoids unnecessary intervention. If it fails, the ablations should reveal whether the monitor is too conservative, too aggressive, or less useful than a static control. The v3 LCB result supports the failure interpretation: monitor-only is the highest observed condition, full DriftGuard ranks behind plain target instruction, and no-gate ties full DriftGuard. The method section therefore defines the mechanism being audited, but it does not claim that this version is the final solution.

The scoring interface is deliberately separated from the controller. Each row records the benchmark family, condition, target language or script, generated output, primary diagnostic bucket, and numeric score. LCB scores target-language adherence for language-confusion prompts. CodeMixQA combines answer containment, target-language behavior, and legitimate code-switching diagnostics. FLORES-200 uses reference-based semantic similarity together with target-script diagnostics. The shared interface lets the same controller be tested across direct adherence, code-switched reasoning, and translation-style semantic preservation.

The workflow has two intended safety properties: minimality, so the gate avoids unnecessary intervention when the model already follows the target-language contract, and specificity, so repair targets language form without rewriting task content. These properties are not assumed to hold; they are exactly what the no-gate, monitor-only, and always-on ablations test. Because no new training is introduced, failures can be attributed to control policy, prompt design, scorer interaction, or model capability rather than to an unreported optimization procedure.

## Experimental Setup

The experiment is a controlled three-family evaluation. It contains 1,920 scored rows: three benchmark families, eight conditions, and 80 tasks per family-condition cell. The selected families are Language Confusion Benchmark, CodeMixQA, and FLORES-200. LCB is the direct language-confusion family; CodeMixQA is the counter-control for legitimate code-switching and answer preservation; FLORES-200 tests semantic preservation under target-language generation. This design keeps direct language adherence, mixed-language answer preservation, and translation-style semantics visible as separate outcomes.

All conditions use the same evaluated model and generation harness: Qwen/Qwen2.5-7B-Instruct through a vLLM backend. Generation uses a fixed maximum budget of 128 new tokens per row in the result matrix. This model choice is not a frontier-model claim. It was selected because the open-weight route exposes the generation behavior needed for a fair train-free baseline comparison under the project budget. The paper therefore treats results as model- and harness-specific evidence about controller behavior, not as a universal statement about all multilingual LLMs.

The eight evaluated conditions are mixed-context baseline, plain target instruction, language-lock prompt, Adaptive LATB, full DriftGuard, DriftGuard without the dynamic gate, monitor-only DriftGuard, and always-on DriftGuard. The primary metric in the main table is mean score, where higher is better within each family. For LCB, this coincides with target-language adherence. For FLORES-200, it reflects semantic/reference score under target-language constraints. For CodeMixQA, it reflects the stricter code-switched answer criterion used by the scorer. Secondary diagnostics include wrong-language rate, truncation rate, safety-template rate, and diagnostic buckets.

Paired comparisons are computed over task-aligned rows. The statistical analysis uses paired sign-flip permutation p-values, paired bootstrap confidence intervals, and Holm correction over the relevant comparisons. We use those tests conservatively. A statistically detectable improvement over a weak mixed-context baseline is not enough to claim that full DriftGuard is useful when plain target instruction or an ablated DriftGuard variant is stronger. The central comparison for positive claims is therefore full DriftGuard against plain target instruction and against the strongest available nontrivial baseline.

Table 1 also explains why the paper does not collapse the benchmark families into a single average. The families do not measure interchangeable outcomes: a direct adherence benchmark can reward intervention, a code-switched question-answering benchmark can penalize the same intervention if it removes necessary mixed-language cues, and a translation benchmark can penalize both wrong language and semantic drift. The evaluation is therefore organized as a family-by-condition matrix, followed by targeted ablations and failure buckets.

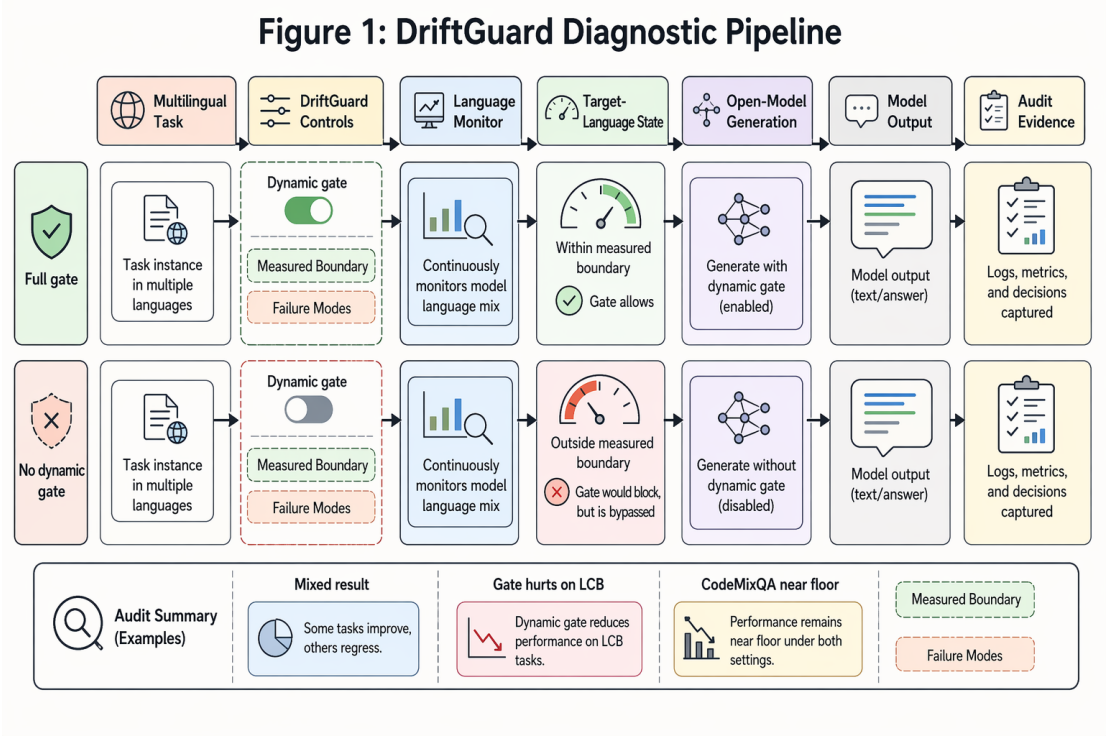


Figure 1: DriftGuard workflow. The monitor-gate-repair loop is the mechanism under test; the reported results show that this version is diagnostic rather than reliably beneficial.

Table 1: The executed setup uses three public benchmark families and 80 tasks per family-condition cell, so each headline comparison is row-paired within a family.

| Family | Primary role | Tasks |
|---|---|---|
| LCB | Target-language adherence | 80 |
| CodeMixQA | Code-switch answer control | 80 |
| FLORES-200 | Translation semantics | 80 |
| Conditions | Eight per family | 640 |
| Total | Scored generation rows | 1,920 |

The sampling policy is deliberately symmetric at the paper level. Each family contributes the same number of task rows to each condition, so differences in the main table are not caused by one condition seeing more examples. This does not remove all uncertainty: the run still has one model, one generation budget, and one sampled slice per family. It does make the first-order comparisons interpretable because differences are computed over aligned task rows rather than unrelated prompt sets.

The conditions are also ordered by increasing mechanism strength. Mixed context asks whether the original benchmark prompt is already enough. Plain target instruction asks how much can be gained by simply naming the output-language requirement. Language lock asks whether a stronger prompt-only self-check helps. Adaptive LATB represents the train-free token-control family that motivated stronger baselines. The DriftGuard variants then ask whether monitoring, gating, and intervention add value over these simpler choices. This order is important because the negative result is not a failure to improve over the weakest prompt; it is a failure to beat the right simple control.

The diagnostic buckets are not post-hoc decorations. They distinguish wrong-language failure from truncation or task failure: an answer that stops early is not a language-control success, even if the visible prefix is in the target script, and a CodeMixQA response that avoids mixing but omits the answer is not preservation. The paper therefore reports mean score with wrong-language and truncation rates, then returns to bucket distributions in the failure analysis.

## Results

Table 2 summarizes the core evidence. The first finding is that the simple prompt baseline is difficult to beat. On LCB, plain target instruction scores 0.8500 while full DriftGuard scores 0.8375, but both LCB rows are length-confounded and have zero measured semantic correctness under the 128-token budget. On FLORES-200, plain target instruction scores 0.446445 while full DriftGuard scores 0.434874. On CodeMixQA, plain target instruction scores 0.0250 while full DriftGuard scores 0.0125, and the family is near floor rather than evidence of preserved code-switched answer quality. This directly contradicts the planned claim that full DriftGuard improves over plain target instruction across all executed families.

The second finding is that DriftGuard no longer gives even a clean weak-baseline win across the completed evaluation matrix. On LCB, full DriftGuard is above mixed-

Table 2: Main score matrix for the completed three-family evaluation. '7B/vLLM' denotes Qwen/Qwen2.5-7B-Instruct with vLLM; budget is maximum new tokens. Full DriftGuard does not clear the plain-prompt comparison on any family.

| Benchmark/source | $N$ | Model | Role | Method | Budget | Metric | Score | $\Delta$ |
|---|---|---|---|---|---|---|---|---|
| LCB | 80 | 7B/vLLM | Base | Mixed | 128 | Adher. | 0.8125 | -0.0375 |
| LCB | 80 | 7B/vLLM | Prompt | Plain | 128 | Adher. | 0.8500 | 0.0000 |
| LCB | 80 | 7B/vLLM | Prompt | Lock | 128 | Adher. | 0.8375 | -0.0125 |
| LCB | 80 | 7B/vLLM | Train-free | LATB | 128 | Adher. | 0.8500 | 0.0000 |
| LCB | 80 | 7B/vLLM | Gated | Full DG | 128 | Adher. | 0.8375 | -0.0125 |
| LCB | 80 | 7B/vLLM | Ablation | No gate | 128 | Adher. | 0.8375 | -0.0125 |
| LCB | 80 | 7B/vLLM | Ablation | Monitor | 128 | Adher. | **0.8625** | +0.0125 |
| CodeMixQA | 80 | 7B/vLLM | Base | Mixed | 128 | Answer | 0.0125 | -0.0125 |
| CodeMixQA | 80 | 7B/vLLM | Prompt | Plain | 128 | Answer | 0.0250 | 0.0000 |
| CodeMixQA | 80 | 7B/vLLM | Prompt | Lock | 128 | Answer | 0.0125 | -0.0125 |
| CodeMixQA | 80 | 7B/vLLM | Train-free | LATB | 128 | Answer | 0.0125 | -0.0125 |
| CodeMixQA | 80 | 7B/vLLM | Gated | Full DG | 128 | Answer | 0.0125 | -0.0125 |
| CodeMixQA | 80 | 7B/vLLM | Ablation | No gate | 128 | Answer | 0.0125 | -0.0125 |
| CodeMixQA | 80 | 7B/vLLM | Ablation | Monitor | 128 | Answer | **0.0375** | +0.0125 |
| FLORES-200 | 80 | 7B/vLLM | Base | Mixed | 128 | Semantic | 0.435585 | -0.010860 |
| FLORES-200 | 80 | 7B/vLLM | Prompt | Plain | 128 | Semantic | 0.446445 | 0.000000 |
| FLORES-200 | 80 | 7B/vLLM | Prompt | Lock | 128 | Semantic | **0.455586** | +0.009140 |
| FLORES-200 | 80 | 7B/vLLM | Train-free | LATB | 128 | Semantic | 0.442856 | -0.003589 |
| FLORES-200 | 80 | 7B/vLLM | Gated | Full DG | 128 | Semantic | 0.434874 | -0.011571 |
| FLORES-200 | 80 | 7B/vLLM | Ablation | No gate | 128 | Semantic | 0.451088 | +0.004643 |
| FLORES-200 | 80 | 7B/vLLM | Ablation | Monitor | 128 | Semantic | 0.436519 | -0.009926 |

context baseline by 0.0250 mean score, but plain target instruction, Adaptive LATB, and monitor-only are all at least as high. On FLORES-200, full DriftGuard is slightly below mixed-context baseline by 0.000710. These values show that adding language-control structure can change behavior, but they do not justify the stronger claim that the full gated method is the best or most robust intervention. The relevant baseline for a paper claim is not an unmodified prompt alone; it is the best simple or published-style control that the method must outperform.

The third finding is that the dynamic gate is not validated by the current LCB ablation. DriftGuard without the dynamic gate and full DriftGuard both reach 0.8375 on LCB, while monitor-only reaches 0.8625; Table 3 gives paired comparisons against the plain prompt. If the gate were beneficial, the full method should beat the simpler ablations under the same budget. Here, it does not. The current evidence does not isolate whether the cause is thresholding, repair wording, or prompt interaction.

Figure 2 visualizes the same family-level pattern. The plot is useful because the rank ordering changes by family. LCB rewards monitor-only most strongly under the adherence score, FLORES-200 favors language-lock prompting, and CodeMixQA is too low to support a positive claim about answer preservation. A single aggregate headline would hide these differences.

The strongest supported statement is therefore comparative and bounded. DriftGuard-style language control exposes measurable differences among prompt, monitor, gate, and repair variants, but the full dynamic controller does not clear the stronger plain-prompt standard and does not validate CodeMixQA preservation. This is a different contribution from a method paper that reports a win rate. It is a diagnostic analysis: the paper identifies which simple comparison falsifies the planned claim and which family exposes the largest unresolved failure.

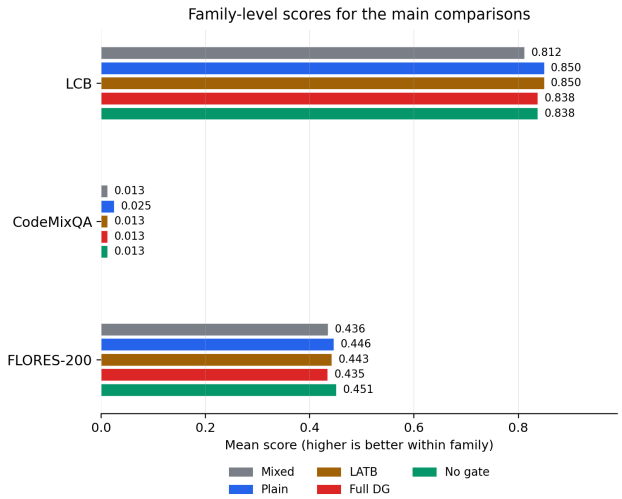


Figure 2: Data-derived family score plot from the completed run. The visual pattern supports a mixed/null interpretation rather than an overall DriftGuard win.

The family-specific results also show why the planned headline could not be rescued by changing the baseline after the fact. On LCB, full DriftGuard improves over mixed context by only 0.0250, while monitor-only improves by 0.0500; on FLORES-200 it is below mixed context and plain target instruction; on CodeMixQA it is below the plain

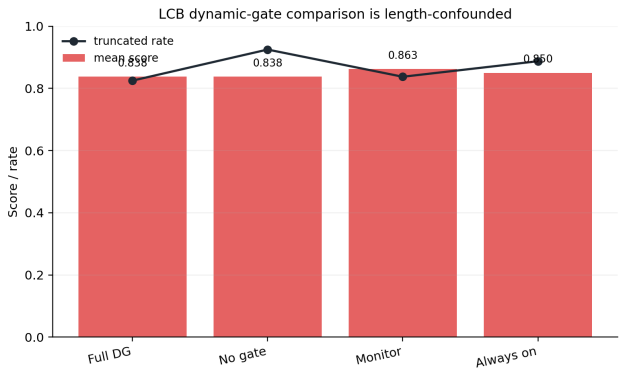


Figure 3: Data-derived LCB dynamic-gate ablation with the same model, backend, and 128-token budget as Table 2. Full DriftGuard does not beat simpler ablations, so the current gate-benefit claim is unsupported.

prompt and near floor. These comparisons leave no defensible route to a broad statement that full DriftGuard is the best evaluated control. A paper that emphasized any single favorable slice would obscure the mechanism failure.

## Analysis and Ablation

The LCB ablations are the clearest evidence about the mechanism, but they are also length-confounded. The no-dynamic-gate ablation and full DriftGuard both score 0.8375, always-on DriftGuard scores 0.8500, and monitor-only reaches 0.8625. The metric is target-language adherence rather than semantic correctness, and truncation is high across LCB conditions. This ranking suggests that the current gate does not identify an intervention boundary that improves the direct confusion slice. The failure is not simply that language control never changes behavior; rather, the full gated recipe does not beat simpler controls under the measured conditions.

The diagnostic rates sharpen the diagnosis. On LCB, wrong-language rate is zero for the displayed methods, but truncation is high: 0.9250 for plain target instruction, 0.8250 for full DriftGuard, and 0.9250 for no dynamic gate. On FLORES-200, wrong-language rates are low but not zero, with plain target instruction at 0.0125 and full DriftGuard at 0.0250. These rates warn that mean score alone is not enough: a method can change truncation, target-script behavior, and semantic score in different directions. Future gate revisions should be optimized against diagnostic buckets rather than a single aggregate score.

CodeMixQA is the clearest rejected claim. Mixed-context baseline, language-lock prompt, Adaptive LATB, full DriftGuard, and no dynamic gate all score 0.0125, plain target instruction scores 0.0250, and monitor-only scores 0.0375. The error analysis records many rows as wrong-language, and answer containment is usually false. Table 4 gives the bucket rates. The key tradeoff is that plain target instruction reduces wrong-language rate relative to mixed context but increases truncation, while full DriftGuard has no truncation in this slice but a higher wrong-language rate. A future version must therefore improve both language-form control and task answer behavior; optimizing only one diagnostic bucket will not turn CodeMixQA into supporting evidence.

The paired tests also favor cautious wording. Full DriftGuard's deficits against plain target instruction on LCB, CodeMixQA, and FLORES-200 are not significant after Holm correction in the v3 table. That lack of significance is not a positive result: the point estimates still fail the planned superiority claim, and the run has a single model and no repeated seeds. These tests estimate row-paired uncertainty rather than model-run variance, so the conclusion avoids claims about robustness across model families or decoding implementations.

These results determine what the paper omits. It does not claim state-of-the-art mitigation, robust superiority over target-language prompting, validation of the dynamic gate, or successful CodeMixQA preservation. It instead claims that the completed matrix identifies a concrete failure of the current gate and exposes CodeMixQA as an unsolved setting for this implementation. This is a narrower but defensible result.

Figure 3 also motivates the next controller design. The monitor should not merely decide whether any language repair is needed; it must decide whether repair is more valuable than preserving the model's current answer trajectory and generation budget. In the current LCB run, the full gate does not improve over no-gate control and does not beat monitor-only. A second analysis point is that truncation and wrong-language behavior move differently: full DriftGuard has lower LCB truncation than plain target instruction, but a worse adherence score and no semantic-correctness evidence, while CodeMixQA shows wrong-language and truncation tradeoffs. Future gates should therefore choose an action only when the expected reduction in wrong-language drift outweighs truncation or semantic risk.

A third point concerns the monitor-only condition. Monitor-only DriftGuard is the highest observed LCB condition and the highest near-floor CodeMixQA condition, but that does not validate DriftGuard as a mitigation method because monitor-only is not the full controller and LCB remains truncation-confounded. The present monitor may still be useful as an audit signal, but the completed run does not show the monitor-gate-repair loop to be a sufficient intervention.

The analysis also separates benchmark failure from method failure. CodeMixQA near-floor behavior may involve prompt formatting, answer extraction, generation budget, or model capability. The available evidence does not justify choosing one cause as the explanation; it only justifies refusing to use CodeMixQA as positive evidence. Likewise, the no-gate ablation is not a replacement policy by itself. Its value is narrower: it shows that the dynamic decision boundary is not adding value on the direct LCB slice, so a revision should test gate thresholds and repair actions against static and monitor-only controls.

## Failure Cases

The failure taxonomy divides errors into wrong-language, truncated, safety-template, legitimate-code-switch,

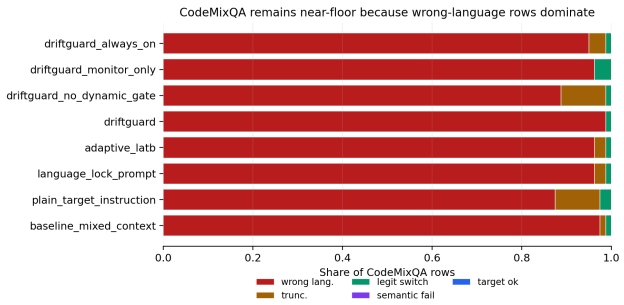


Figure 4: Data-derived CodeMixQA diagnostic buckets. Wrong-language and truncation failures dominate, while legitimate code-switching is nearly absent in the scored outputs.

target-language-ok, and semantic-failure buckets. On CodeMixQA, wrong-language behavior dominates many conditions, while truncation is also common. These are not minor formatting artifacts: they change the interpretation of any language-control score. A controller that shortens or truncates outputs may appear to avoid wrong-language spans while failing the answer task; a controller that enforces a target language may erase code-switches that are expected by the benchmark row.

One recurrent CodeMixQA pattern is answer-language mismatch. The model often responds in a fluent language that is not aligned with the row's requested mixed-language form, or it repeats the question form without producing the answer string. Budget failures also appear when responses are cut before the answer can be evaluated. These patterns show that the current setup does not yet separate controller failure from answer-extraction and generation-budget failure.

LCB failure cases are different. The no-gate variant and full DriftGuard tie on target-language adherence while still carrying high truncation rates. Full DriftGuard reduces truncation relative to plain target instruction on LCB but does not produce the best mean score. This tradeoff indicates that future work should report length and truncation jointly with target-language adherence. A method that wins by generating short, incomplete answers would not solve the user-facing problem.

FLORES-200 failure cases emphasize semantic preservation. The family uses reference translations, so a target-script-looking answer can still receive a low semantic score. Language-lock prompting has the best displayed mean score, while plain target instruction also beats full DriftGuard. DriftGuard's lower score and higher wrong-language rate show that the current repair policy does not reliably convert target-language reminders into better translations. This makes FLORES-200 a useful guard against overfitting to surface-script adherence.

These failure cases are useful precisely because they disagree. LCB says that stronger intervention may help adherence, CodeMixQA says the current answer setting is too brittle for preservation claims, and FLORES-200 says semantic score and target-language diagnostics can move separately. For LCB, the next ablation is therefore narrow: compare no-gate and monitor-only variants against full DriftGuard under gate-threshold sweeps. That sweep is not part of the current evidence, so the present paper only reports that the fixed gate did not beat simpler ablations.

The interaction between these families explains why the negative result belongs in the main body. The combined matrix gives a more useful diagnosis than any family alone: target-language pressure, semantic preservation, and code-switch preservation are separable axes, and the current controller has not found a policy that improves all three. A future positive claim would need to beat the LCB monitor-only and no-gate controls without increasing truncation, then repair or replace the CodeMixQA path so that legitimate code-switching is observable.

A final practical failure case is reporting bias. The evaluation contains one tempting positive slice: full DriftGuard is slightly above mixed context on LCB. That slice is true, but it is not sufficient for the planned contribution because plain target instruction, Adaptive LATB, always-on, and monitor-only are at least as strong on the same family. The plain prompt and ablation comparisons are more informative because they test whether the extra mechanism is necessary. We therefore report favorable slices only with their disqualifying comparisons nearby.

## Scope and Limitations

The study is scoped to one open-weight model, one vLLM backend, one 128-new-token budget, and one sampled slice of 80 tasks per family-condition cell. Its paired tests estimate row-level uncertainty within that matrix rather than model-run variance across seeds, decoding implementations, or model families. LCB remains length-confounded, so its adherence scores cannot be read as semantic success; CodeMixQA remains near floor, so it cannot support a preservation claim. These limits are part of the result: the paper reports a diagnostic failure of the present gate and a concrete specification for future controls, not a universal mitigation guarantee.

## Conclusion

The completed evidence matrix does not support a broad positive DriftGuard claim. Full DriftGuard trails plain target instruction on CodeMixQA and FLORES-200, remains length-confounded on LCB, and does not beat simpler DriftGuard ablations. The actionable result is diagnostic: a future controller must clear plain target-language prompting, language-lock prompting, monitor-only behavior, and static no-gate intervention under the same evaluation budget, while reporting wrong-language rate, truncation rate, semantic score, and paired uncertainty together. A stronger claim should be restored only after a revised gate improves on those controls and CodeMixQA produces observable answer-preservation evidence rather than near-floor scores.

## References


Abdaljalil, S.; Serpedin, E.; Qaraqe, K.; and Kurban, H. 2025. Evaluating Multilingual and Code-Switched Alignment in LLMs via Synthetic Natural Language Inference. ArXiv preprint, arXiv:2508.14735.

Aguilar, G.; Kar, S.; and Solorio, T. 2020. LinCE: A Centralized Benchmark for Linguistic Code-switching Evaluation. ArXiv preprint, arXiv:2005.04322.

Ai, X.; Ihsani, M. K.; and Kan, M.-Y. 2025. Are Knowledge and Reference in Multilingual Language Models Cross-Lingually Consistent? ArXiv preprint, arXiv:2507.12838.

Asano, S.; Baek, J.; and Yamasaki, T. 2026. Beyond Bilingual Transfer: Multilingual Code-Switching in Instruction Tuning. ArXiv preprint, arXiv:2605.29414.

Blevins, T.; Gonen, H.; and Zettlemoyer, L. 2022. Analyzing the Mono- and Cross-Lingual Pretraining Dynamics of Multilingual Language Models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.

Chen, Y.; Biswas, R.; Lent, H.; and Bjerva, J. 2024a. Against All Odds: Overcoming Typology, Script, and Language Confusion in Multilingual Embedding Inversion Attacks. ArXiv preprint, arXiv:2408.11749.

Chen, Y.; Li, Q.; Biswas, R.; and Bjerva, J. 2024b. Large Language Models are Easily Confused: A Quantitative Metric, Security Implications and Typological Analysis. ArXiv preprint, arXiv:2410.13237.

Choo, J.; Lee, J.; Kim, J.; Song, Y.; Hong, S. K.; and Kwon, Y.-D. 2026. TLPO: Token-Level Policy Optimization for Mitigating Language Confusion in Large Language Models. ArXiv preprint, arXiv:2604.26553.

Conneau, A.; Khandelwal, K.; Goyal, N.; Chaudhary, V.; Wenzek, G.; Guzman, F.; Grave, E.; Ott, M.; Zettlemoyer, L.; and Stoyanov, V. 2019. Unsupervised Cross-lingual Representation Learning at Scale. ArXiv preprint, arXiv:1911.02116.

Deng, B.; Wan, Y.; Zhang, Y.; Yang, B.; and Feng, F. 2025. Unveiling Language-Specific Features in Large Language Models via Sparse Autoencoders. ArXiv preprint, arXiv:2505.05111.

Devlin, J.; Chang, M.-W.; Lee, K.; and Toutanova, K. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. ArXiv preprint, arXiv:1810.04805.

Dumas, C.; Wendler, C.; Veselovsky, V.; Monea, G.; and West, R. 2024. Separating Tongue from Thought: Activation Patching Reveals Language-Agnostic Concept Representations in Transformers. ArXiv preprint, arXiv:2411.08745.

Goncharov, A.; Kondusov, N.; and Zaytsev, A. 2025. Language steering in latent space to mitigate unintended code-switching. ArXiv preprint, arXiv:2510.13849.

Gurgurov, D.; Al Ghussin, Y.; Baeumel, T.; Chou, C.-T.; Schramowski, P.; Mosbach, M.; van Genabith, J.; and Ostermann, S. 2026. CLaS-Bench: A Cross-Lingual Alignment and Steering Benchmark. ArXiv preprint, arXiv:2601.08331.

Ji, Y.; Wang, J.; Li, J.; Ye, H.; and Zhang, M. 2023. Isotropic Representation Can Improve Zero-Shot Cross-Lingual Transfer on Multilingual Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

Kaji, A.; and Shah, M. 2023. Contextual Code Switching for Machine Translation using Language Models. ArXiv preprint, arXiv:2312.13179.

Kirtane, N.; and Huang, K.-H. 2026. Language Steering for Multilingual In-Context Learning. ArXiv preprint, arXiv:2602.02326.

Lample, G.; and Conneau, A. 2019. Cross-lingual Language Model Pretraining. ArXiv preprint, arXiv:1901.07291.

Lee, N.; Woo, Y.; Ko, H.; and Son, G. 2025. Controlling Language Confusion in Multilingual LLMs. ArXiv preprint, arXiv:2505.19116.

Li, C.; Wang, S.; Zhang, J.; and Zong, C. 2023. Improving In-context Learning of Multilingual Generative Language Models with Cross-lingual Alignment. ArXiv preprint, arXiv:2311.08089.

Lopo, J. A.; Habibi, M. R. S.; Wong, T. H.; Ghozali, M. I.; Koto, F.; Winata, G. I.; Limkonchotiwat, P.; Aji, A. F.; and Cahyawijaya, S. 2025. Language Surgery in Multilingual Large Language Models. ArXiv preprint, arXiv:2506.12450.

Mahmoud, O.; Semage, B. L.; Karimpanal, T. G.; and Rana, S. 2025. Improving Multilingual Language Models by Aligning Representations through Steering. ArXiv preprint, arXiv:2505.12584.

Marchisio, K.; Ko, W.-Y.; Berard, A.; Dehaze, T.; and Ruder, S. 2024. Understanding and Mitigating Language Confusion in LLMs. ArXiv preprint, arXiv:2406.20052.

Nie, E.; Schmid, H.; and Schutze, H. 2025. Mechanistic Understanding and Mitigation of Language Confusion in English-Centric Large Language Models. ArXiv preprint, arXiv:2505.16538.

Park, J.; Yoon, S.; Jun, Y.; and Lee, H. 2026. Code-Switching Reveals Language Anchoring in Multilingual LLMs. ArXiv preprint, arXiv:2606.19668.

Philippy, F.; Guo, S.; Klein, J.; and Bissyande, T. F. 2026. When English Is Not the Best Teacher: Source Language Effects in Cross-Lingual In-Context Learning. ArXiv preprint, arXiv:2606.18033.

Ravisankar, K.; Han, H.; Wiegreffe, S.; and Carpuat, M. 2025. Can You Map It to English? The Role of Cross-Lingual Alignment in Multilingual Performance of LLMs. ArXiv preprint, arXiv:2504.09378.

Saji, A.; Husain, J. A.; Jayakumar, T.; Dabre, R.; Kunchukuttan, A.; and Puduppully, R. 2025. RomanLens: The Role of Latent Romanization in Multilinguality in LLMs. ArXiv preprint, arXiv:2502.07424.

Sarapat, P.; Ukarapol, T.; and Hashimoto, T. 2025. Language Confusion and Multilingual Performance: A Case Study of Thai-Adapted Large Language Models. In *Proceedings of the 1st Workshop on Confabulation, Hallucinations and Overgeneration in Multilingual and Practical Settings*.

Sterz, H.; Schmidt, F. D.; Glavas, G.; and Vulic, I. 2025. ReCoVeR the Target Language: Language Steering without Sacrificing Task Performance. ArXiv preprint, arXiv:2509.14814.

Ukarapol, T.; Sarapat, P.; and Chukamphaeng, N. 2026. Language-Aware Token Boosting: LLM Language Confusion Reduction Without Tuning. ArXiv preprint, arXiv:2606.08994.

Wang, Z.; Li, J.; Zhou, H.; Weng, R.; Wang, J.; Huang, X.; Han, X.; Feng, J.; Deng, C.; and Huang, S. 2025. Investigating and Scaling up Code-Switching for Multilingual Language Model Pre-Training. ArXiv preprint, arXiv:2504.01801.

Winata, G. I.; Anugraha, D.; Irawan, P. A.; Das, A.; Yoo, H.; Dashore, P.; Kulkarni, S.; Zhang, R.; Sakajo, H.; Hudi, F.; Ovalle, A.; Montariol, S.; Gaschi, F.; Anugraha, M.; Puranik, R. R.; Ahmed, Z. H.; Merin, A. P.; and Chersoni, E. 2026. Can Large Language Models Understand, Reason About, and Generate Code-Switched Text? ArXiv preprint, arXiv:2601.07153.

Wong, S. H.; Sajjad, H.; and Siddique, A. B. 2026. LangFIR: Discovering Sparse Language-Specific Features from Monolingual Data for Language Steering. ArXiv preprint, arXiv:2604.03532.

Xue, L.; Constant, N.; Roberts, A.; Kale, M.; Al-Rfou, R.; Siddhant, A.; Barua, A.; and Raffel, C. 2020. mT5: A massively multilingual pre-trained text-to-text transformer. ArXiv preprint, arXiv:2010.11934.

Yoo, H.; Yang, Y.; and Lee, H. 2024. Code-Switching Red-Teaming: LLM Evaluation for Safety and Multilingual Understanding. ArXiv preprint, arXiv:2406.15481.

Zhang, R.; Cahyawijaya, S.; Cruz, J. C. B.; Winata, G. I.; and Aji, A. F. 2023. Multilingual Large Language Models Are Not (Yet) Code-Switchers. ArXiv preprint, arXiv:2305.14235.

Zhong, C.; Cheng, F.; Liu, Q.; Murawaki, Y.; Chu, C.; and Kurohashi, S. 2025. Language Lives in Sparse Dimensions: Toward Interpretable and Efficient Multilingual Control for Large Language Models. ArXiv preprint, arXiv:2510.07213.

## Reproducibility Checklist

**Data and benchmarks.** The study uses three public benchmark families: Language Confusion Benchmark, CodeMixQA, and FLORES-200. The executed matrix contains 80 tasks per family-condition cell and 1,920 scored rows. The benchmark descriptions and licenses should be cited with the public dataset documentation in any released package.

**Model and backend.** The evaluated model is Qwen/Qwen2.5-7B-Instruct through a vLLM generation backend. The paper does not claim frontier-scale behavior. The generation budget recorded in the result matrix is 128 new tokens per row.

**Conditions.** The evaluated conditions are mixed-context baseline, plain target instruction, language-lock prompt, Adaptive LATB, full DriftGuard, DriftGuard without dynamic gate, monitor-only DriftGuard, and always-on DriftGuard. All paper claims are tied to the completed evaluation.

**Metrics and statistics.** Primary scores, wrong-language rates, truncation rates, paired permutation tests, paired bootstrap intervals, and Holm-corrected p-values are generated from the scored rows. The paper reports row-paired uncertainty and explicitly does not estimate model-run variance.

**Reconstruction package.** A reproducibility release should include the prompts or prompt templates, scored outputs, summary tables, metric scripts, paired-test scripts, model identifier, generation budget, and figure-generation scripts for data plots. Conceptual diagrams are explanatory and are not part of the evaluated system.

**Claim calibration.** Contradicted or rejected hypotheses are not used as contribution claims. The paper treats them as findings or failure analysis.

## Appendix: Additional Diagnostic Tables

Table 3 reports the paired comparisons behind the main diagnostic interpretation. The hypothesis summary is compact enough to state in prose: H1, broad improvement over plain target instruction, is contradicted; H2, improvement over Adaptive LATB, is contradicted by the selected v3 rows; H3, dynamic-gate superiority on LCB, is contradicted; and H4, CodeMixQA preservation, is rejected.

Table 3: Paired comparisons against the plain prompt show why the current result is diagnostic: full DriftGuard has negative point estimates on all three families and no Holm-significant win.

| Comparison | Family | Delta | Holm p |
|---|---|---|---|
| DG - plain | LCB | -0.0125 | 1.0000 |
| DG - plain | FLORES | -0.011571 | 1.0000 |
| DG - plain | CodeMixQA | -0.0125 | 1.0000 |

Table 4 gives the CodeMixQA bucket rates referenced in the failure-case discussion. It is retained as a table because the wrong-language and truncation tradeoff differs by condition and cannot be summarized by the near-floor mean score alone.

Table 4: CodeMixQA diagnostic rates show why near-floor scores are not usable evidence for code-switch preservation.

| Condition | Wrong lang. | Trunc. | Code-switch |
|---|---|---|---|
| Mixed context | 0.9750 | 0.0125 | 0.0125 |
| Plain prompt | 0.8750 | 0.1000 | 0.0250 |
| Adaptive LATB | 0.9625 | 0.0250 | 0.0125 |
| Full DG | 0.9875 | 0.0000 | 0.0125 |
| No gate | 0.8875 | 0.1000 | 0.0125 |
| Monitor | 0.9625 | 0.0000 | 0.0375 |

## Appendix: Additional Diagnostic Notes

The current matrix suggests three repair directions. First, the dynamic gate needs calibration against no-dynamic-gate and monitor-only ablations, because the full controller is not

stronger on LCB. Second, CodeMixQA requires a targeted prompt, answer-extraction, and budget analysis before it can support preservation claims. Third, future runs should include repeated seeds or repeated model samples if the paper needs uncertainty over generation variability rather than only paired row-level uncertainty.